

1. Funzioni in C++

♦ 1.1 Definizione e dichiarazione

- Cos'è una funzione
- Differenza tra dichiarazione e definizione

Sintassi base:

```
tipo_di_ritorno nome_funzione(parametri) {  
    // corpo della funzione  
}
```

♦ 1.2 Parametri delle funzioni

- Passaggio per valore
- Passaggio per riferimento
- Passaggio per puntatore
- Parametri con valore di default

♦ 1.3 Tipi di ritorno

- Tipi primitivi (int, double, char, ecc.)
- `void` (assenza di ritorno)
- Puntatori e riferimenti
- Array e strutture

♦ 1.4 Prototipi di funzione

- Necessità del prototipo (soprattutto per l'ordine nel codice)
- Dichiarazione nel file header `.h`

♦ 1.5 Funzioni ricorsive

- Definizione e funzionamento
- Esempi classici: fattoriale, Fibonacci
- Condizione di terminazione

♦ 1.6 Overloading (sovraccarico)

- Stessa funzione con parametri diversi
- Regole per l'overloading valido

♦ 1.7 Scope e visibilità

- Variabili locali e globali
- Durata e visibilità delle variabili
- Parola chiave `static`

2. Manipolazione dei file in C++

♦ 2.1 Libreria `fstream`

Inclusione della libreria:

```
#include <fstream>
```

♦ 2.2 Tipi di stream

- `ifstream` – input da file
- `ofstream` – output su file
- `fstream` – input/output

♦ 2.3 Apertura e chiusura dei file

Apertura:

```
ifstream inFile("nomefile.txt");  
ofstream outFile("nomefile.txt");
```

Chiusura:

```
file.close();
```

♦ 2.4 Modalità di apertura

- Modalità predefinita (`ios::in`, `ios::out`)
- Altre modalità:
 - `ios::app` – append
 - `ios::trunc` – sovrascrittura
 - `ios::binary` – modalità binaria

Combinazione delle modalità:

```
ofstream file("nomefile.txt", ios::out | ios::app);
```

♦ 2.5 Lettura e scrittura

- Lettura da file:
 - Con `>>` (estrazione)
 - Con `getline()`
- Scrittura su file:
 - Con `<<`
 - Con `write()` per file binari

♦ 2.6 Controllo degli errori

Verifica dell'apertura:

```
if (!file.is_open()) { /* gestisci errore */ }
```

- Controllo dello stato:
 - `eof()` – fine file
 - `fail()` – errore di I/O
 - `bad()` – errore irreversibile
 - `good()` – tutto OK

♦ 2.7 Posizionamento del cursore

- Funzioni per file testuali e binari:
 - `seekg()`, `tellg()` per input
 - `seekp()`, `tellp()` per output
- Utili per leggere/scrivere da/a posizioni specifiche

♦ 2.8 File binari

- Apertura in modalità binaria

Uso di `read()` e `write()`:

```
file.write(reinterpret_cast<char*>(&oggetto), sizeof(Oggetto));
```

2.9 Esempio

Testo

Esempio di codice C++ che apre un file di testo, legge il suo contenuto riga per riga e lo copia in un secondo file.

```
#include <iostream>

#include <fstream>

#include <string>

int main() {

    std::ifstream inputFile("origine.txt");        // File da leggere

    std::ofstream outputFile("copia.txt");          // File dove
    scrivere la copia

    // Verifica se i file sono stati aperti correttamente

    if (!inputFile.is_open()) {

        std::cerr << "Errore: impossibile aprire il file di
origine.\n";

        return 1;

    }

    if (!outputFile.is_open()) {

        std::cerr << "Errore: impossibile creare il file di
destinazione.\n";

        return 1;

    }

}
```

```

    }

    std::string riga;

    // Legge riga per riga e scrive nel file di destinazione
    while (std::getline(inputFile, riga)) {
        outputFile << riga << '\n';
    }

    // Chiude i file
    inputFile.close();
    outputFile.close();

    std::cout << "Copia completata con successo.\n";

    return 0;
}

```

Spiegazione:

- `ifstream inputFile("origine.txt")`: apre il file in sola lettura.
- `ofstream outputFile("copia.txt")`: crea o sovrascrive il file di output.
- `getline(inputFile, riga)`: legge ogni riga del file.
- `outputFile << riga << '\n'`: scrive la riga nel nuovo file, mantenendo l'andata a capo.

- `inputFile.close()` e `outputFile.close()`: chiudono i file dopo l'uso.

Suggerimenti per lo studio

- 💡 Inizia con funzioni semplici, poi sperimenta con il passaggio per riferimento e ricorsione.
- 📄 Fai pratica con file testuali prima di passare ai file binari.
- 🔍 Prova a scrivere un piccolo progetto (es. rubrica o diario) che usa funzioni e file.